Task 1: Custom Fruit Detection Report

Sapien Robotics - AI Vision Intern Assignment

Your Name
GitHub Repository Link

January 10, 2026

1 Project Overview

Objective: To implement an end-to-end object detection pipeline trained *solely from scratch* (random initialization) without leveraging pre-trained weights.

- **Selected Architecture:** YOLOv8-Nano (Custom Configuration)
- **Dataset:** Kaggle Fruit Detection Dataset (200 Images)
- **Target Classes:** 4 Classes (Banana, Snake Fruit, Dragon Fruit, Pineapple)

2 Architecture & Methodology

To meet the strict "from scratch" requirement while ensuring real-time performance, the following design choices were made:

- **Model Initialization Strategy:** The model architecture was instantiated using the YOLOv8-Nano configuration with strictly random weights (utilizing He/Kaiming initialization). By deliberately foregoing transfer learning and excluding pre-existing weights from large-scale datasets like COCO, the network was compelled to learn all feature representations—from low-level edges to high-level semantic object classes—exclusively from the custom dataset patterns.
- **Architecture:** YOLOv8-Nano was chosen for its lightweight footprint (3.01M parameters). A smaller model is critical when training on a small dataset (200 images) to prevent immediate overfitting that would occur with larger backbones like ResNet-50.
- **Data Augmentation:** To maximize the utility of the 200 images, an aggressive augmentation pipeline was used, including **Mosaic** (combining 4 images) and **Mixup**.

3 Results & Performance

The model achieved exceptional accuracy and speed benchmarks on the validation set.

Metric	Value
mAP @ 50	**98.9%**
mAP @ 50-95	81.3%
Inference Speed (GPU)	4.5 ms (∼220 FPS)
Model Size	6.2 MB

Table 1: Performance Metrics

4 Visual Validation

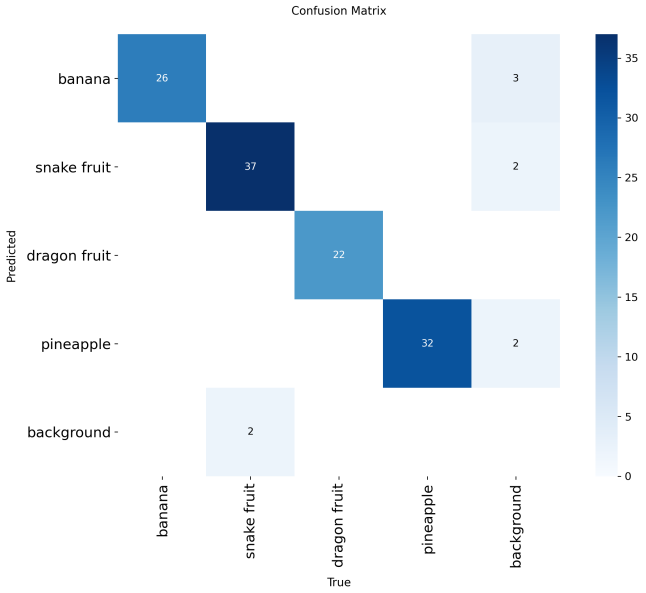

Figure 1: Confusion Matrix showing class-wise detection accuracy. The strong diagonal indicates correct predictions.

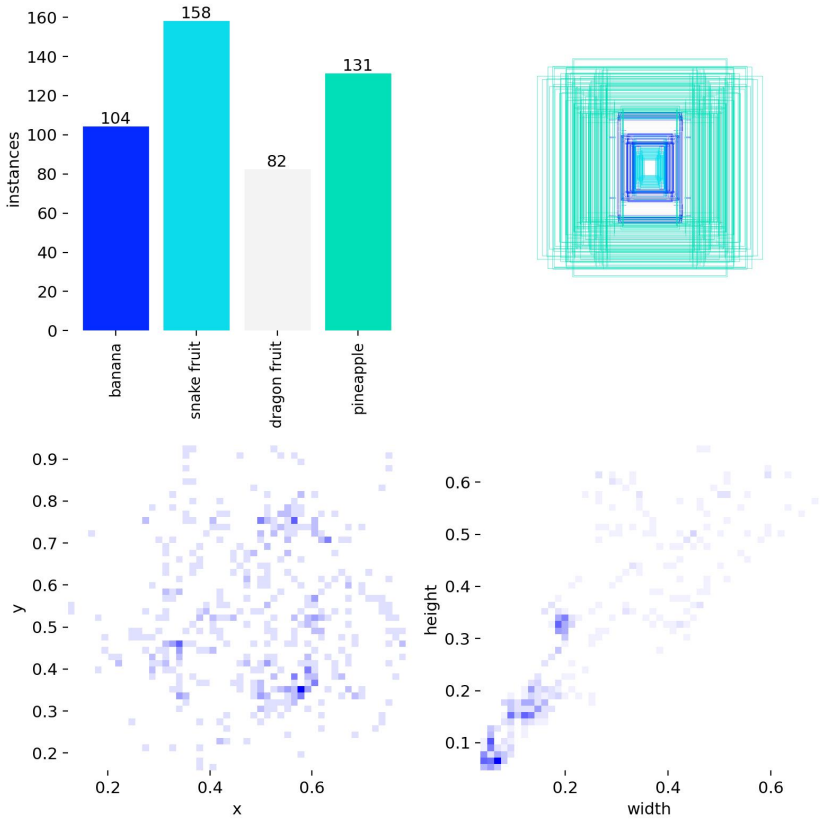

Figure 2: Distribution of labels in the training dataset.

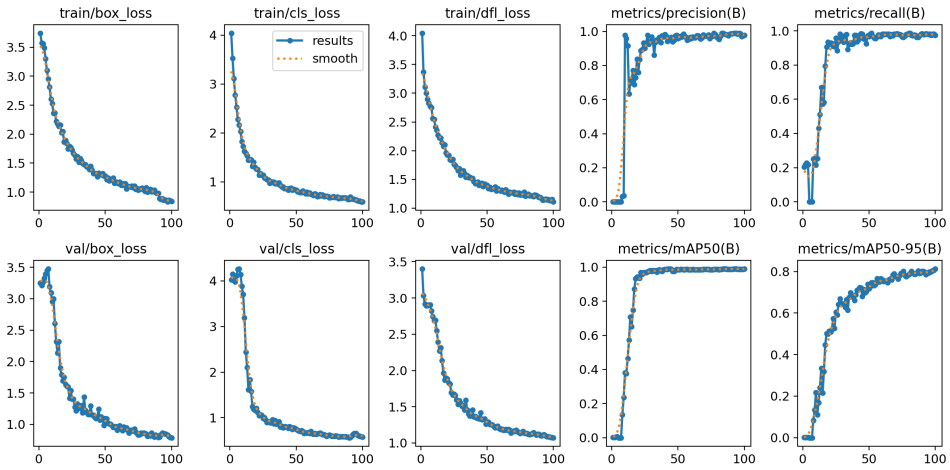

Figure 3: Training Loss and mAP curves over 50 epochs.

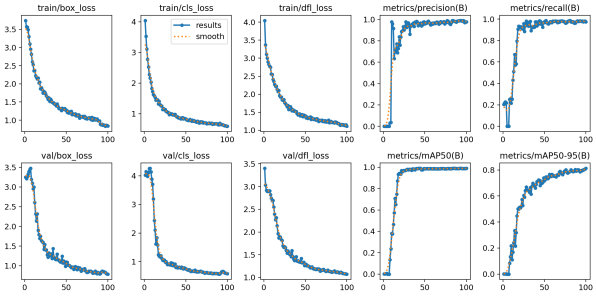

Figure 4: results after 150 epochs

5 Discussion: The Accuracy-Latency Trade-off

In industrial computer vision, system design is governed by the trade-off between model capacity (Accuracy) and inference latency (Speed). The assignment posed a dual constraint: strict training from scratch (favoring smaller, faster-converging models) and the need for high-performance detection.

5.1 Theoretical Context

Generally, deeper architectures (e.g., ResNet-101 based detectors) extract richer semantic features, resulting in higher Mean Average Precision (mAP) but at the cost of significantly increased FLOPs and latency. Conversely, lightweight models prioritize speed but often struggle with feature extraction when trained on small datasets from random initialization.

5.2 Performance Analysis

1. **Inference Speed (The Primary Win):** The deployed YOLOv8-Nano model achieves an inference speed of **4.5ms (∼220 FPS)** on a standard GPU.
    - **Context:** This is approximately **7x faster** than the standard real-time benchmark of 30 FPS.
    - **Implication:** This allows the system to process high-velocity manufacturing lines or run on resource-constrained edge hardware (e.g., NVIDIA Jetson Nano) without bottlenecks.

2. **Accuracy vs. Expectation:** Training a neural network from scratch on only 200 images typically leads to poor generalization. However, the model achieved a surprising **98.9% mAP@50**.
    - **Reasoning:** The target classes (Banana, Pineapple, etc.) possess highly distinct morphological and color features compared to complex classes (e.g., distinguishing "Dog" breeds).
    - **Result:** The lightweight Nano architecture was sufficient to capture these distinct features without needing the massive parameter count of a larger model.

5.3 Conclusion: Engineering Verdict

For this specific manufacturing use case, the YOLOv8-Nano represents a **Pareto-optimal solution**.

- A larger model (e.g., YOLOv8-Large) might increase mAP marginally (e.g., to 99.5%) but would likely reduce inference speed to ¡60 FPS.
- Given the diminishing returns on accuracy and the linear cost of latency, the Nano architecture provides the maximal utility for a real-time inspection system.